

Sommaire

1	Introduction	3
2	Méthodes agiles	4
2.1	Genèse de la méthodologie agile	4
2.1.1	Des origines industrielles au Lean Manufacturing	4
2.1.2	La crise du logiciel dans les années 1990	4
2.1.3	Le manifeste agile	5
2.2	Les valeurs du Manifeste Agile	5
2.2.1	Les individus et leurs interactions	5
2.2.2	Un logiciel opérationnel	5
2.2.3	La collaboration avec le client	5
2.2.4	L'adaptation au changement	6
2.3	Les principes de l'agilité	6
3	Framework SCRUM	7
3.1	Présentation	7
3.1.1	Définition et origines	7
3.1.2	Les valeurs de Scrum	7
3.1.3	Caractéristiques générales de Scrum	8
3.2	Les rôles dans la méthodologie Scrum	8
3.3	Les artefacts de la méthodologie Scrum	9
3.4	Les cérémonies Scrum	9
3.4.1	Sprint Planning	9
3.4.2	Daily Scrum :	10
3.4.3	Sprint Review :	10
3.4.4	Sprint Retrospective :	10
3.5	Avantages et limites de Scrum	10
3.5.1	Les avantages de Scrum	10
3.5.2	Les limites de Scrum	11
3.6	Comparaison entre Scrum et d'autres méthodes de gestion de projet	11
3.7	Scrum et la méthode itérative	12
3.8	Scrum et le modèle en cascade	12
3.9	Scrum et le cycle en V	12
3.10	Tableau récapitulatif comparatif	13
4	Application de la méthodologie Scrum au projet MINIP	13
4.1	Présentation du contexte du projet	13
4.2	Problèmes identifiés et solution proposée	13
4.3	Constitution de l'équipe Scrum	14
4.4	Élaboration du Product Backlog	14
4.5	Planification et déroulement des Sprints	16
4.5.1	Sprint 1 : Gestion des stocks et de la caisse (Semaines 3 et 4)	16
4.5.2	Sprint 2 : Gestion des employés, paiements mobiles et système antivol (Semaines 5 et 6)	16
4.6	Déroulement des cérémonies Scrum	16

4.7	Bilan de l'application de Scrum au projet MINIP	17
5	Conclusion	18
6	Déclaration d'utilisation de l'intelligence artificielle	19

1 Introduction

Dans un monde économique où les technologies et les marchés évoluent à un rythme très rapide, la capacité d'adaptation des entreprises est devenue une condition essentielle pour durer. Pendant longtemps, la gestion de projet informatique s'est appuyée sur des modèles traditionnels et rigides, comme le modèle en cascade . Cette approche classique exigeait de planifier la totalité du projet dès le départ : l'écriture détaillée des besoins, la conception, le développement, puis les tests à la toute fin. Cependant, cette méthode s'est souvent révélée inefficace pour les logiciels. Elle entraîne régulièrement des retards, des dépassements de budget et livre parfois des produits qui ne correspondent plus aux attentes des clients, car leurs besoins ont changé entre-temps.

Pour répondre à ces difficultés, les méthodes agiles ont été créées. Leur principe fondamental est d'accepter le changement en cours de route comme une opportunité plutôt que comme un problème. Au lieu de figer les besoins au début du projet, l'équipe avance pas à pas.

Parmi les différents cadres de travail agiles, Scrum est aujourd'hui le plus populaire et le plus utilisé à travers le monde. Il propose une structure collaborative simple basée sur des rôles précis, des réunions régulières et une visibilité totale sur l'avancement du projet.

L'objectif de ce travail est de présenter les grands principes des méthodologies agiles, de comprendre leur origine historique, puis d'analyser comment le framework Scrum permet de mener à bien des projets complexes.

2 Méthodes agiles

2.1 Genèse de la méthodologie agile

Les méthodes agiles sont apparues en réponse aux limites observées dans les approches traditionnelles de gestion de projet appliquées au développement logiciel. En effet, les méthodes issues du secteur industriel, bien qu'efficaces pour la production de biens matériels, se sont révélées peu adaptées à un domaine caractérisé par l'incertitude, l'évolution constante des besoins et la nécessité d'une forte collaboration entre les différents acteurs du projet.

2.1.1 Des origines industrielles au Lean Manufacturing

Au début du XX^e siècle, les organisations industrielles étaient fortement influencées par le modèle tayloriste. Cette approche reposait sur une séparation stricte entre la conception, assurée par les ingénieurs, et l'exécution, réalisée par les ouvriers. Avec l'émergence de l'informatique, plusieurs organisations ont tenté d'appliquer ce modèle au développement logiciel. Cependant, contrairement à la fabrication de produits standardisés, la conception d'un logiciel nécessite des activités de réflexion, de créativité et d'adaptation permanentes.

Dans les années 1950, l'entreprise japonaise Toyota introduit une nouvelle philosophie de production connue sous le nom de *Lean Manufacturing*. Cette approche vise principalement à améliorer l'efficacité des processus tout en maximisant la valeur apportée au client. Elle repose notamment sur trois principes fondamentaux :

- l'élimination des activités n'apportant pas de valeur ajoutée ;
- la production juste à temps en fonction de la demande réelle ;
- l'autonomisation des équipes afin de favoriser la prise de décision au plus près du terrain.

Quelques décennies plus tard, en 1986, les chercheurs japonais Hirotaka Takeuchi et Ikujiro Nonaka publient un article intitulé *The New New Product Development Game*. Dans leurs travaux, ils observent que les entreprises les plus performantes s'appuient sur des équipes multidisciplinaires travaillant de manière collaborative plutôt que séquentielle. Ils comparent cette organisation à une équipe de rugby avançant collectivement vers un objectif commun en se transmettant continuellement le ballon. C'est dans ce contexte que le terme *Scrum*, qui signifie « mêlée » en anglais, est utilisé pour la première fois afin de décrire cette nouvelle approche du travail en équipe.

2.1.2 La crise du logiciel dans les années 1990

Au cours des années 1990, l'industrie du logiciel connaît une période marquée par de nombreux échecs de projets. Les dépassements de budget, les retards de livraison ainsi que les abandons de projets deviennent fréquents. Cette situation est notamment mise en évidence par le *Chaos Report* publié en 1994, qui révèle qu'une faible proportion des projets informatiques parvenait à être achevée dans les délais et les budgets prévus.

Face à ce constat, plusieurs experts du développement logiciel commencent à élaborer des approches plus flexibles, mieux adaptées à la nature évolutive des projets informatiques. Parmi les initiatives les plus marquantes figurent :

- Scrum, formalisé et présenté en 1995 par Jeff Sutherland et Ken Schwaber ;

- Extreme Programming (XP), développé par Kent Beck, qui met l'accent sur les bonnes pratiques de développement logiciel et la qualité du code.

Ces approches, qualifiées à l'époque de méthodes légères (*Lightweight Methods*), cherchent à réduire la rigidité des méthodes traditionnelles et à favoriser une meilleure adaptation aux changements.

2.1.3 Le manifeste agile

L'année 2001 constitue une étape déterminante dans l'évolution des méthodes agiles. En février de cette année, dix-sept spécialistes du développement logiciel se réunissent à Snowbird, dans l'État de l'Utah aux États-Unis, afin de partager leurs expériences et d'harmoniser leurs différentes approches.

Cette rencontre aboutit à la rédaction du **Manifeste pour le développement agile de logiciels** (*Agile Manifesto*), considéré aujourd'hui comme l'acte fondateur du mouvement agile. Ce manifeste énonce un ensemble de quatre valeurs et douze principes afin de faciliter le processus de développement logiciel.

2.2 Les valeurs du Manifeste Agile

Le Manifeste Agile ne définit pas une méthode de développement unique. Il établit plutôt un ensemble de valeurs communes destinées à guider les équipes dans leur manière de concevoir et de réaliser des projets logiciels. Ces valeurs mettent en avant certains aspects du développement sans pour autant nier l'importance des autres. Elles constituent le socle sur lequel reposent les différentes méthodes agiles.

2.2.1 Les individus et leurs interactions

La première valeur du Manifeste Agile privilégie les individus et leurs interactions par rapport aux processus et aux outils. Elle repose sur l'idée que la réussite d'un projet dépend avant tout des personnes qui y participent et de leur capacité à collaborer efficacement. Une communication directe, la confiance mutuelle et le partage des connaissances favorisent davantage l'atteinte des objectifs qu'une application rigide de procédures prédéfinies. Les outils et les processus demeurent importants, mais ils doivent être considérés comme des moyens au service de l'équipe et non comme une finalité.

2.2.2 Un logiciel opérationnel

La deuxième valeur met l'accent sur la livraison d'un logiciel fonctionnel plutôt que sur la production d'une documentation exhaustive. Dans les approches agiles, la valeur apportée au client se matérialise principalement à travers un produit utilisable répondant à ses besoins. Bien que la documentation reste nécessaire pour assurer la maintenance et la compréhension du système, elle ne doit pas ralentir la livraison des fonctionnalités. L'objectif est donc de produire régulièrement des versions opérationnelles du logiciel afin de recueillir rapidement des retours d'expérience.

2.2.3 La collaboration avec le client

La troisième valeur privilégie la collaboration avec le client plutôt que la négociation contractuelle. Les méthodes agiles considèrent le client comme un acteur essentiel du projet, impliqué tout au long du processus de développement. Sa participation régulière permet de clarifier les besoins, de

redéfinir les priorités lorsque cela est nécessaire et de valider les fonctionnalités développées. Cette collaboration continue réduit les risques d'incompréhension et contribue à la réalisation d'un produit davantage conforme aux attentes des utilisateurs.

2.2.4 L'adaptation au changement

La quatrième valeur met en avant l'adaptation au changement plutôt que le suivi strict d'un plan initial. Les projets logiciels évoluent dans un environnement où les besoins, les contraintes et les technologies peuvent changer rapidement. Les méthodes agiles considèrent donc ces changements comme des opportunités d'amélioration plutôt que comme des perturbations. Cette capacité d'adaptation permet aux équipes de répondre plus efficacement aux nouvelles exigences et d'accroître la valeur du produit développé.

2.3 Les principes de l'agilité

En complément des quatre valeurs fondamentales, le Manifeste Agile définit douze principes qui précisent les pratiques et les comportements à adopter pour mettre en œuvre l'agilité au sein des équipes de développement. Ces principes sont les suivants :

1. Satisfaire le client en livrant rapidement et continuellement des logiciels à forte valeur ajoutée.
2. Accueillir favorablement les changements de besoins, même à un stade avancé du développement.
3. Livrer fréquemment un logiciel opérationnel, de préférence toutes les deux à quatre semaines.
4. Assurer une collaboration quotidienne entre les experts métier et les développeurs.
5. Construire les projets autour d'individus motivés en leur fournissant l'environnement et le soutien nécessaires à leur réussite.
6. Privilégier la communication directe comme moyen le plus efficace de transmettre l'information.
7. Considérer le logiciel opérationnel comme la principale mesure de l'avancement du projet.
8. Maintenir un rythme de développement soutenable sur le long terme.
9. Porter une attention constante à l'excellence technique et à la qualité de la conception.
10. Rechercher la simplicité en minimisant le travail inutile.
11. Favoriser les équipes auto-organisées, dont émergent les meilleures architectures et conceptions.
12. Encourager l'amélioration continue à travers une réflexion régulière sur les pratiques de l'équipe afin d'accroître son efficacité.

Ces principes constituent le cadre de référence des méthodes agiles. Ils orientent les équipes vers une démarche centrée sur la création de valeur, la collaboration entre les parties prenantes et l'amélioration continue des processus de développement.

3 Framework SCRUM

3.1 Présentation

3.1.1 Définition et origines

Scrum est un cadre de travail (*framework*) agile conçu pour accompagner les équipes dans le développement et la maintenance de produits complexes au sein d'environnements caractérisés par des changements fréquents. Développé au début des années 1990 par Ken Schwaber et Jeff Sutherland, Scrum ne constitue pas une méthodologie prescriptive définissant précisément chaque étape du processus de développement. Il propose plutôt un ensemble de pratiques et de règles permettant aux équipes de s'adapter efficacement aux évolutions des besoins et aux contraintes du projet.

Ce cadre de travail repose principalement sur l'empirisme et les principes du Lean. Il privilégie l'apprentissage à partir de l'expérience, l'observation continue des résultats obtenus et l'amélioration progressive des processus. Ainsi, les décisions sont prises sur la base d'informations réelles et observables plutôt que sur des prévisions établies à long terme.

Les piliers de Scrum Le fonctionnement de Scrum repose sur trois piliers fondamentaux qui permettent de garantir la maîtrise du projet et la création continue de valeur.

La transparence La transparence consiste à rendre visibles et compréhensibles tous les aspects importants du projet pour l'ensemble des parties prenantes. Les objectifs, les exigences, l'état d'avancement des travaux ainsi que les éventuels obstacles doivent être clairement communiqués. Cette transparence favorise une meilleure collaboration et facilite la prise de décision tout au long du développement.

L'inspection L'inspection correspond à l'évaluation régulière des artefacts Scrum ainsi que de l'avancement du projet. Cette vérification fréquente permet d'identifier rapidement les écarts entre les résultats obtenus et les objectifs fixés. Elle contribue ainsi à détecter précocement les risques, les défauts ou les difficultés susceptibles d'affecter la qualité du produit.

L'adaptation L'adaptation constitue la conséquence directe de l'inspection. Lorsqu'un problème est identifié ou qu'un changement intervient dans l'environnement du projet, l'équipe doit ajuster son organisation, ses priorités ou le produit en cours de développement afin de maintenir sa capacité à atteindre les objectifs définis. Cette aptitude à s'adapter rapidement représente l'un des principaux avantages de Scrum.

3.1.2 Les valeurs de Scrum

Au-delà de ses pratiques, Scrum repose également sur cinq valeurs fondamentales qui orientent le comportement des membres de l'équipe et favorisent un environnement de travail efficace.

- **L'engagement** : chaque membre de l'équipe s'investit pleinement dans l'atteinte des objectifs du Sprint et contribue activement à la réussite collective.
- **Le courage** : les membres de l'équipe font preuve de courage dans leurs prises de décision, dans la résolution des problèmes et dans la gestion des situations complexes.

- **La focalisation** : l'équipe concentre ses efforts sur les objectifs du Sprint en cours afin de maximiser la valeur produite.
- **L'ouverture** : les parties prenantes communiquent de manière transparente sur l'avancement du projet, les difficultés rencontrées et les résultats obtenus.
- **Le respect** : chaque membre est considéré comme un professionnel compétent et autonome dont les compétences et les contributions sont reconnues par l'ensemble de l'équipe.

3.1.3 Caractéristiques générales de Scrum

Scrum possède plusieurs caractéristiques qui le distinguent des approches traditionnelles de gestion de projet.

Tout d'abord, il adopte une approche itérative et incrémentale. Le développement est organisé en cycles de courte durée appelés *Sprints*, généralement compris entre une et quatre semaines. Chaque Sprint regroupe l'ensemble des activités nécessaires à la réalisation d'une partie du produit, notamment la planification, la conception, le développement et les tests.

À l'issue de chaque Sprint, l'équipe produit un incrément de produit fonctionnel. Cet incrément correspond à une version du logiciel respectant la *Definition of Done* et pouvant être utilisée ou évaluée par le client.

Scrum favorise également l'auto-organisation des équipes. Les développeurs disposent d'une autonomie importante pour choisir les solutions techniques et organiser leur travail de manière à atteindre les objectifs fixés.

Par ailleurs, une collaboration continue est maintenue avec les utilisateurs et les clients, généralement représentés par le Product Owner. Cette proximité facilite la prise en compte des retours d'expérience et permet d'adapter rapidement le produit aux besoins réels.

Enfin, Scrum encourage une démarche d'amélioration continue. À la fin de chaque Sprint, l'équipe analyse son fonctionnement lors de la rétrospective afin d'identifier les points à améliorer et de mettre en place des actions correctives pour les itérations suivantes.

3.2 Les rôles dans la méthodologie Scrum

La méthodologie Scrum repose sur la collaboration de plusieurs acteurs ayant des responsabilités bien définies. Trois rôles principaux sont essentiels à la mise en œuvre efficace de cette approche agile.

- **Product Owner** : Le Product Owner est responsable de la définition et de la valorisation du produit. Il identifie les besoins des clients, des utilisateurs et du marché afin de déterminer les fonctionnalités à développer en priorité. Pour cela, il gère et maintient le *Product Backlog*, qui regroupe l'ensemble des besoins du projet. Il veille également au respect des exigences du client et constitue l'interface principale entre les parties prenantes et l'équipe de développement.
- **Scrum Master** : Le Scrum Master est le garant de l'application des principes et des pratiques Scrum. Il accompagne l'équipe dans l'adoption de la méthodologie agile, facilite la communication entre les différents acteurs et veille à la suppression des obstacles susceptibles de ralentir l'avancement du projet. Il est également chargé d'organiser et d'animer les différents événements Scrum, tels que la planification de sprint, les réunions quotidiennes (*Daily Scrum*), les revues de sprint et les rétrospectives.

- **Équipe de développement** : L'équipe de développement regroupe les professionnels chargés de réaliser les tâches définies pour chaque sprint afin de produire les livrables attendus. Généralement constituée de cinq à sept membres aux compétences complémentaires, elle est responsable de la conception, du développement, des tests et de la livraison des fonctionnalités prévues.

3.3 Les artefacts de la méthodologie Scrum

Les artefacts Scrum représentent les éléments de travail et les informations utilisées par l'équipe pour planifier, suivre et évaluer l'avancement du projet. Ils assurent la transparence et facilitent la prise de décision tout au long du développement. Scrum définit trois artefacts principaux : le Product Backlog, le Sprint Backlog et l'incrément de produit.

- **Product Backlog** : Le Product Backlog est une liste évolutive et priorisée regroupant l'ensemble des fonctionnalités, exigences, améliorations et corrections à apporter au produit. Ces éléments sont généralement formulés sous forme de **User Stories**. Le Product Backlog est continuellement mis à jour et réorganisé par le Product Owner en fonction des besoins du projet et des retours des parties prenantes.
- **Sprint Backlog** : Le Sprint Backlog correspond à l'ensemble des éléments du Product Backlog sélectionnés pour être réalisés au cours d'un sprint. Il inclut également les tâches nécessaires à leur implémentation. Défini par l'équipe lors de la réunion de planification du sprint, il peut être ajusté en fonction des contraintes et des découvertes réalisées durant le cycle de développement.
- **Incrément de produit** : L'incrément de produit représente la version du produit obtenue à l'issue d'un sprint. Il regroupe toutes les fonctionnalités achevées et validées durant le sprint, auxquelles s'ajoutent les incréments produits lors des sprints précédents. Cet incrément doit être fonctionnel, conforme à la définition de terminé (*Definition of Done*) et potentiellement livrable aux utilisateurs.

3.4 Les cérémonies Scrum

Les cérémonies Scrum, également appelées événements time-boxés, constituent les principaux points de synchronisation qui structurent chaque Sprint. Elles permettent de rythmer le travail de l'équipe tout en assurant la transparence du projet, l'inspection régulière de l'avancement et l'adaptation continue du plan de développement en fonction des retours et des contraintes rencontrées.

3.4.1 Sprint Planning

La Sprint Planning correspond à la réunion de planification du Sprint. Elle a pour objectif de définir clairement le but du Sprint ainsi que de déterminer et organiser le travail nécessaire pour atteindre cet objectif. Lors de cette cérémonie, l'équipe sélectionne les éléments du Product Backlog à traiter et les transforme en tâches concrètes à réaliser.

Cette réunion réunit le Scrum Master, le Product Owner et les développeurs. Pour un Sprint d'une durée d'un mois, elle est limitée à un maximum de huit heures.

3.4.2 Daily Scrum :

Le Daily Scrum est une réunion quotidienne de synchronisation de l'équipe de développement. Elle permet d'évaluer l'avancement du travail et d'ajuster le plan d'action pour les 24 heures à venir. L'objectif principal est de favoriser la coordination entre les membres de l'équipe et d'identifier rapidement les éventuels obstacles.

Cette cérémonie est principalement destinée aux développeurs, tandis que le Scrum Master intervient en tant que facilitateur si nécessaire. Elle est limitée à 15 minutes et se tient à heure et lieu fixes chaque jour.

3.4.3 Sprint Review :

La Sprint Review est une réunion organisée à la fin de chaque Sprint. Elle a pour objectif de présenter l'incrément de produit réalisé aux parties prenantes et de recueillir leurs retours. Ces échanges permettent d'orienter les prochaines étapes du développement en fonction des attentes et des besoins exprimés.

Cette cérémonie rassemble l'équipe Scrum ainsi que les parties prenantes invitées par le Product Owner. Sa durée est limitée à un maximum de quatre heures pour un Sprint d'un mois.

3.4.4 Sprint Retrospective :

La Sprint Retrospective constitue un moment d'analyse et de réflexion sur le fonctionnement de l'équipe. Elle vise à examiner les processus, les interactions et les outils utilisés afin d'identifier les points d'amélioration et de définir des actions concrètes pour le Sprint suivant.

Cette réunion concerne l'ensemble de l'équipe Scrum, avec la participation obligatoire du Scrum Master et des développeurs. La présence du Product Owner est optionnelle. Sa durée est limitée à un maximum de trois heures pour un Sprint d'un mois.

3.5 Avantages et limites de Scrum

Scrum est un cadre de travail agile largement utilisé dans le développement logiciel en raison de sa flexibilité et de sa capacité à s'adapter à des environnements complexes et évolutifs. Toutefois, malgré ses nombreux avantages, il présente également certaines limites qu'il convient de prendre en compte afin d'assurer une mise en œuvre efficace.

3.5.1 Les avantages de Scrum

- **Transparence du projet** : Scrum offre une visibilité constante sur l'avancement du projet grâce aux cérémonies régulières (Daily Scrum, Sprint Review) et aux artefacts (Product Backlog, Sprint Backlog). Cette transparence facilite la communication entre les membres de l'équipe et permet une meilleure prise de décision.
- **Adaptabilité aux changements** : Le fonctionnement itératif de Scrum permet de réévaluer les priorités à la fin de chaque Sprint. Cela facilite l'intégration des retours des utilisateurs et l'ajustement du produit en fonction de l'évolution des besoins.
- **Amélioration de la productivité** : Le principe de time-boxing permet de structurer le travail en périodes courtes et définies, limitant ainsi les réunions inutiles et favorisant la concentration sur les objectifs du Sprint.

- **Amélioration continue** : Les rétrospectives encouragent une analyse régulière des pratiques de l'équipe afin d'identifier les points d'amélioration et de mettre en place des actions correctives pour les Sprints suivants.
- **Collaboration et responsabilisation** : Scrum favorise la communication directe, l'auto-organisation et la responsabilisation des membres de l'équipe, ce qui renforce la cohésion et l'efficacité collective.

3.5.2 Les limites de Scrum

- **Dépendance à la maturité de l'équipe** : L'efficacité de Scrum repose fortement sur la discipline, la communication et la maturité des membres de l'équipe. Sans ces éléments, le cadre peut perdre en efficacité.
- **Vision long terme parfois limitée** : La focalisation sur les objectifs à court terme (Sprint) peut réduire la visibilité stratégique globale du produit si elle n'est pas correctement équilibrée.
- **Pression sur les équipes** : Une mauvaise planification ou une mauvaise animation des cérémonies peut entraîner une surcharge de travail ou une pression excessive sur les membres de l'équipe.
- **Scalabilité limitée** : Scrum est particulièrement adapté aux petites équipes. Son utilisation à grande échelle nécessite souvent des frameworks complémentaires pour assurer la coordination entre plusieurs équipes.

3.6 Comparaison entre Scrum et d'autres méthodes de gestion de projet

Le choix d'une méthode de gestion de projet dépend fortement de la nature du produit à développer, du niveau de stabilité des besoins ainsi que du degré d'incertitude présent dans l'environnement. Afin de mieux situer Scrum dans le paysage des méthodes de développement logiciel, il est pertinent de le comparer à plusieurs approches classiques du génie logiciel, notamment :

- **Le modèle en cascade** : Il s'agit d'une approche séquentielle traditionnelle dans laquelle le projet est découpé en phases successives (analyse, conception, développement, test, déploiement). Chaque phase doit être entièrement achevée avant de passer à la suivante, avec une forte importance accordée à la documentation.
- **Le cycle en V** : Il constitue une évolution du modèle en cascade. Il introduit une correspondance directe entre les phases de conception et les phases de validation et de test. Cette approche met l'accent sur la traçabilité des exigences et le contrôle qualité tout au long du cycle de développement.
- **La méthode itérative** : Elle repose sur le développement progressif du produit à travers des cycles successifs appelés itérations. Chaque itération permet d'améliorer et d'enrichir le système en fonction des retours et des apprentissages obtenus, sans nécessairement imposer un cadre organisationnel strict comme dans les méthodes agiles.

3.7 Scrum et la méthode itérative

Scrum peut être considéré comme une forme avancée de méthode itérative, mais toutes les méthodes itératives ne sont pas nécessairement agiles. En effet, une approche itérative classique se concentre principalement sur la construction progressive du produit, souvent sans cadre organisationnel formalisé.

Scrum, quant à lui, va plus loin en structurant non seulement le développement, mais également l'organisation de l'équipe. Il impose des règles précises telles que l'auto-organisation des équipes, la durée fixe des Sprints et la mise en place de cérémonies régulières (Sprint Planning, Daily Scrum, Sprint Review et Sprint Retrospective). De plus, chaque cycle doit produire un incrément potentiellement livrable, ce qui renforce la notion de valeur ajoutée continue.

3.8 Scrum et le modèle en cascade

La comparaison entre Scrum et le modèle en cascade met en évidence une opposition forte dans la philosophie de gestion de projet. Le modèle en cascade repose sur une planification complète et détaillée en amont du projet. Chaque étape doit être validée avant de passer à la suivante, ce qui rend les modifications en cours de projet difficiles et coûteuses.

Scrum adopte une approche opposée en intégrant l'incertitude comme un élément naturel du développement logiciel. Grâce à son fonctionnement itératif et incrémental, il permet de développer, tester et ajuster le produit de manière continue sur des cycles courts. Cela réduit l'effet tunnel et garantit une meilleure adéquation entre le produit final et les besoins réels des utilisateurs.

3.9 Scrum et le cycle en V

Le cycle en V constitue une méthode prédictive qui repose sur une planification rigoureuse et une documentation exhaustive dès les premières phases du projet. Chaque phase de développement est associée à une phase de test correspondante, ce qui assure une forte traçabilité et un contrôle qualité structuré.

Cependant, cette rigidité limite fortement la capacité d'adaptation aux changements. Scrum, à l'inverse, privilégie la flexibilité et l'adaptation continue. Il permet une livraison rapide et incrémentale du produit, facilitant ainsi l'intégration des retours utilisateurs.

Néanmoins, le cycle en V conserve sa pertinence dans des contextes où les exigences sont stables et fortement réglementées, tels que les domaines de l'aéronautique, de la défense ou du médical, où la conformité et la traçabilité sont des critères essentiels.

3.10 Tableau récapitulatif comparatif

Critère	Scrum	Méthode itérative	Cycle en cascade	Cycle en V
Philosophie	Agile, itérative et empirique	Évolutive basée sur l'apprentissage	Linéaire et prédictive	Prédictive avec forte traçabilité
Cadence temporelle	Sprints à durée fixe	Cycles successifs de durée variable	Phases séquentielles rigides	Phases séquentielles structurées
Gestion du changement	Acceptée à chaque Sprint	Intégrée entre les itérations	Très difficile et coûteuse	Difficile et formalisée
Organisation	Auto-organisation des équipes	Organisation variable	Hierarchique classique	Hierarchique avec rôles spécialisés
Principaux bénéfices	Adaptabilité, livraison rapide, collaboration	Flexibilité progressive	Simplicité contractuelle initiale	Traçabilité et contrôle qualité

Table 1: Tableau comparatif des méthodes de gestion de projet

4 Application de la méthodologie Scrum au projet MINIP

4.1 Présentation du contexte du projet

Afin d'illustrer concrètement la mise en œuvre de la méthodologie Scrum, cette section présente son application dans le cadre d'un projet réel : le développement d'un système informatique de gestion pour la supérette MINIP.

MINIP est un supermarché proposant à sa clientèle une large gamme de produits du quotidien, notamment dans les domaines de l'alimentation, des boissons, de l'hygiène et des produits ménagers. Face à une croissance progressive de son activité, la direction de l'établissement a décidé de moderniser ses processus opérationnels en dotant le magasin d'une application informatique complète, intégrant un système antivol intelligent basé sur la reconnaissance par code-barres.

4.2 Problèmes identifiés et solution proposée

Avant le lancement du projet, MINIP souffrait de plusieurs dysfonctionnements opérationnels susceptibles de nuire à la performance globale de l'entreprise. Ces difficultés concernaient notamment la gestion des stocks, marquée par des ruptures non anticipées et des commandes redondantes, ainsi que des erreurs récurrentes en caisse liées à des prix incorrects ou à des remises mal appliquées. Des pertes inexplicables dues à des écarts entre les stocks théoriques et réels, une traçabilité insuffisante des actions des employés, et l'absence d'indicateurs de performance pour une gestion avisée du commerce.

Afin de répondre à ces besoins, le projet consiste à développer une application desktop de gestion structurée autour de cinq modules fonctionnels principaux : la gestion des produits et des stocks, la gestion des ventes et de la caisse, la gestion des employés, un système antivol intelligent à portiques de détection, ainsi qu'un module de rapports et de tableau de bord. Sur le plan technologique, la solution repose sur React pour l'interface utilisateur, Laravel pour la logique métier côté serveur, et PostgreSQL comme système de gestion de base de données.

4.3 Constitution de l'équipe Scrum

Conformément aux principes de la méthodologie Scrum, une équipe structurée autour des trois rôles fondamentaux a été constituée pour piloter le projet MINIP.

- **Product Owner** : Ce rôle est assuré par le gérant de MINIP. En tant que principal représentant des parties prenantes, il est chargé de définir les besoins prioritaires, de maintenir le Product Backlog à jour et de valider les fonctionnalités livrées à l'issue de chaque Sprint. Sa proximité avec les opérations quotidiennes du magasin lui confère une connaissance approfondie des exigences métier.
- **Scrum Master** : Ce rôle est occupé par HOUÉHA Karen, également développeuse senior au sein du projet. Elle est responsable de l'application rigoureuse des pratiques Scrum, de la facilitation des cérémonies et de la levée des obstacles pouvant ralentir l'avancement de l'équipe.
- **Équipe de développement** : Elle est composée de quatre développeurs aux compétences complémentaires : AVALLA Onésime, AWOUDO Nelson, ADJOVI Ruben, AGBADJAGAN Stella et MUSUWAR Amatul. Conformément aux principes de Scrum, cette équipe est auto-organisée et responsable de l'ensemble des activités de conception, de développement et de test des fonctionnalités.

4.4 Élaboration du Product Backlog

Le Product Backlog constitue la liste exhaustive et priorisée de l'ensemble des fonctionnalités à développer pour le projet MINIP. Chaque élément y est exprimé sous la forme d'une *User Story*, formulée selon le modèle suivant : « *En tant que [rôle], je veux [action] afin de [bénéfice]*. »

Le Product Backlog du projet MINIP regroupe dix-huit User Stories réparties en cinq modules . Ces éléments ont été priorisés par le Product Owner en fonction de leur importance et de leur impact opérationnel, puis répartis sur trois Sprints. Le tableau suivant en présente une synthèse :

ID	User Story	Module	Priorité	Effort (pts)	Sprint
US-01	En tant que gérant, je veux ajouter un produit au catalogue afin de gérer l'inventaire.	Stocks	Haute	5	Sprint 1
US-02	En tant que magasinier, je veux modifier la quantité d'un produit après livraison.	Stocks	Haute	3	Sprint 1
US-03	En tant que gérant, je veux définir un seuil d'alerte par produit.	Stocks	Haute	3	Sprint 1
US-04	En tant que caissier, je veux scanner le code-barres d'un produit en caisse.	Ventes	Haute	5	Sprint 1
US-05	En tant que caissier, je veux calculer automatiquement le total et la monnaie à rendre.	Ventes	Haute	3	Sprint 1
US-06	En tant que caissier, je veux imprimer un ticket de caisse après chaque vente.	Ventes	Haute	3	Sprint 1
US-07	En tant qu'administrateur, je veux créer des comptes utilisateurs avec des rôles distincts.	Employés	Haute	5	Sprint 2
US-08	En tant que gérant, je veux consulter les présences de mes employés.	Employés	Moyenne	3	Sprint 2
US-09	En tant que caissier, je veux accepter les paiements Mobile Money.	Ventes	Haute	5	Sprint 2
US-10	En tant que caissier, je veux enregistrer un retour de produit.	Ventes	Moyenne	3	Sprint 2
US-11	En tant qu'agent de sécurité, je veux que le portique vérifie les articles non payés.	Antivol	Haute	8	Sprint 2
US-12	En tant qu'agent de sécurité, je veux recevoir une alerte en temps réel.	Antivol	Haute	5	Sprint 2
US-13	En tant qu'agent de sécurité, je veux consulter les détails d'une alerte.	Antivol	Haute	5	Sprint 2
US-14	En tant que gérant, je veux consulter le chiffre d'affaires par période.	Rapports	Haute	5	Sprint 3
US-15	En tant que gérant, je veux lister les produits en rupture ou en alerte de stock.	Rapports	Haute	3	Sprint 3
US-16	En tant que gérant, je veux accéder au journal des incidents de sécurité.	Rapports	Moyenne	5	Sprint 3
US-17	En tant que gérant, je veux consulter l'historique des actions de chaque employé.	Rapports	Moyenne	3	Sprint 3
US-18	En tant qu'administrateur, je veux que les données soient sauvegardées automatiquement.	Sécurité	Haute	5	Sprint 3
Total				78 pts	3 Sprints

Table 2: Product Backlog du projet MINIP

4.5 Planification et déroulement des Sprints

Le développement du projet MINIP a été organisé en trois Sprints successifs, dont les deux premiers font l'objet de la présente simulation. Ces deux Sprints, d'une durée de deux semaines chacun, couvrent ainsi quatre semaines de développement actif. Cette organisation permet une livraison progressive et contrôlée des fonctionnalités, tout en maintenant une visibilité constante sur l'avancement du projet.

4.5.1 Sprint 1 : Gestion des stocks et de la caisse (Semaines 3 et 4)

Le premier Sprint a pour objectif principal de livrer les fonctionnalités fondamentales permettant à MINIP de gérer ses produits, ses stocks et ses opérations de caisse. Ces fonctionnalités représentent les besoins les plus urgents identifiés par le Product Owner, dans la mesure où elles conditionnent directement le bon fonctionnement quotidien du magasin.

À l'issue de ce Sprint, l'équipe doit être en mesure de fournir un incrément fonctionnel intégrant les six User Stories sélectionnées (US-01 à US-06), représentant un total de 22 points d'effort. La *Definition of Done* retenue stipule que chaque User Story doit être développée, testée unitairement, revue par le Product Owner et déployée sur l'environnement de test.

4.5.2 Sprint 2 : Gestion des employés, paiements mobiles et système antivol (Semaines 5 et 6)

Le second Sprint, qui constitue le dernier cycle présenté dans cette simulation, vise à enrichir le système en intégrant les modules complémentaires, notamment la gestion des employés, les paiements via Mobile Money ainsi que le système antivol à portiques. Grâce à la composition de l'équipe, l'intégration du système antivol est réalisée en parallèle du développement des autres modules, optimisant ainsi l'utilisation des ressources disponibles.

Ce Sprint regroupe sept User Stories (US-07 à US-13), pour un effort total estimé à 29 points. La *Definition of Done* de ce Sprint exige que les fonctionnalités soient intégrées et validées sur l'environnement de test, que les scénarios antivol soient testés et que les rôles et permissions soient opérationnels. Les fonctionnalités de reporting ainsi que les exigences de sécurité des données, regroupées dans le troisième Sprint, n'entrent pas dans le périmètre de la présente simulation.

4.6 Déroulement des cérémonies Scrum

Les cérémonies Scrum constituent les points de synchronisation qui structurent chaque Sprint. Elles ont été appliquées de manière rigoureuse tout au long du projet MINIP afin d'assurer la transparence, l'inspection régulière de l'avancement et l'adaptation continue du plan de développement.

- **Sprint Planning** : Organisée en début de chaque Sprint pour une durée de deux à trois heures, cette réunion réunit l'ensemble de l'équipe Scrum ainsi que le gérant de MINIP. Elle permet de sélectionner les User Stories à réaliser, de les décomposer en tâches concrètes et de les estimer en termes d'effort. À l'issue de cette cérémonie, l'équipe dispose d'un Sprint Backlog clair et partagé.
- **Daily Scrum** : Tenue chaque matin du lundi au vendredi pour une durée maximale de quinze minutes, cette réunion quotidienne permet à chaque développeur de partager l'avancement de ses travaux, d'annoncer ses objectifs pour la journée et de signaler les éventuels obstacles rencontrés. Elle favorise ainsi la coordination de l'équipe et la détection précoce des difficultés.

- **Sprint Review** : Organisée en fin de Sprint pour une durée d'une à deux heures, cette cérémonie permet de présenter au gérant de MINIP l'incrément de produit réalisé. Lors du second Sprint, par exemple, la démonstration a porté sur le système de paiement Mobile Money, le fonctionnement des portiques antivols et les rapports de sécurité. Les retours recueillis orientent ensuite la priorisation des prochaines itérations.
- **Sprint Retrospective** : Tenue après la Sprint Review pour une durée d'environ une heure, cette réunion offre à l'équipe un espace de réflexion sur son fonctionnement. Elle vise à identifier ce qui a bien fonctionné, les points d'amélioration et les actions correctives à mettre en place dès le Sprint suivant. Cette cérémonie incarne concrètement le principe d'amélioration continue propre à la méthodologie Scrum.

4.7 Bilan de l'application de Scrum au projet MINIP

L'application de la méthodologie Scrum au projet MINIP illustre de manière concrète les bénéfices d'une approche agile dans un contexte de développement logiciel à contraintes réelles. La structuration du projet en Sprints courts a permis de livrer progressivement des fonctionnalités opérationnelles, réduisant ainsi les risques liés à une livraison unique en fin de projet.

5 Conclusion

La gestion de projet logiciel constitue un domaine en constante évolution, dont les pratiques se sont profondément transformées au cours des dernières décennies. Face aux limites des approches traditionnelles, marquées par leur rigidité et leur incapacité à s'adapter aux changements inévitables des besoins, les méthodes agiles ont progressivement imposé une nouvelle manière de concevoir et de piloter les projets informatiques.

Le Manifeste Agile, publié en 2001, a constitué le point de départ d'une véritable révolution culturelle dans le domaine du développement logiciel. En plaçant les individus, la collaboration et l'adaptation au changement au cœur de ses valeurs, il a posé les fondements d'une approche centrée sur la création de valeur continue pour le client. Parmi les différents cadres de travail issus de ce mouvement, Scrum s'est imposé comme le plus répandu, en proposant une structure légère mais rigoureuse, articulée autour de rôles précis, d'artefacts transparents et de cérémonies régulières.

L'analyse théorique du framework Scrum a permis de mettre en évidence ses principaux atouts : la transparence qu'il instaure, sa capacité à intégrer le changement à chaque itération, la responsabilisation des équipes qu'il favorise et l'amélioration continue qu'il encourage. Cette analyse a également permis de situer Scrum par rapport aux approches classiques telles que le modèle en cascade et le cycle en V, soulignant les contextes dans lesquels chaque méthode conserve sa pertinence.

De plus l'application de Scrum au projet MINIP a permis de concrétiser ces principes théoriques dans un environnement réel.

En définitive, Scrum ne constitue pas une solution universelle applicable à tout type de projet. Son efficacité repose avant tout sur la maturité de l'équipe, l'implication réelle du Product Owner et une adhésion sincère aux valeurs agiles. Cependant, dans des contextes caractérisés par l'évolution des besoins et la nécessité d'une livraison rapide de valeur, il représente un cadre de travail particulièrement adapté, dont l'adoption contribue significativement à la réussite des projets logiciels complexes.

6 Déclaration d'utilisation de l'intelligence artificielle

La réalisation de ce rapport a fait appel à des outils d'intelligence artificielle à des étapes précises et délimitées du processus de travail.

Dans un premier temps, chaque membre du groupe a rédigé de manière autonome une première version de sa section respective, en s'appuyant sur des sources disponibles en ligne ainsi que sur sa propre compréhension des concepts abordés. Cette phase initiale de rédaction a été réalisée intégralement sans recours à l'intelligence artificielle, afin de garantir une appropriation personnelle et rigoureuse des contenus traités.

Dans un second temps, une fois ces premières versions établies, l'intelligence artificielle a été sollicitée afin d'accompagner la mise au propre rédactionnelle de chaque section. Cette intervention a porté principalement sur la reformulation, la fluidité du style et la cohérence de l'expression écrite, sans modifier le fond ni les idées développées par les auteurs. Les textes ainsi améliorés ont ensuite été relus et affinés par les membres du groupe en vue de leur intégration dans le document commun.

Il convient par ailleurs de préciser que, dans le cadre de la section consacrée à la simulation de l'application de la méthodologie Scrum au projet MINIP, l'intelligence artificielle a également été mobilisée pour contribuer à la définition et à la formulation des User Stories constituant le Product Backlog.

Enfin, lors de la phase de mise en commun des différentes contributions, et après correction et sélection du contenu final retenu, l'intelligence artificielle a été utilisée afin d'harmoniser le style de rédaction de l'ensemble du document. Cette dernière intervention visait à assurer une cohérence stylistique et terminologique globale, sans altérer le contenu ni les choix rédactionnels propres à chaque section.